Yenepoya Institute of
Arts, Science, Commerce and Management
Project – Low Level Design
on
Natural Language interface for EdTech infrastructure

Student Name: Fathima Sherin

Industry Mentor:

Guided by: MS Manjula

Table of Contents

1. Introduction. 6
   1.1. Scope of the document. 6
   1.2. Intended Audience 6
   1.3. System overview. 6

2. System Design. 6
   2.1. Application Design 6
   2.2. Process Flow. 6
   2.3. Information Flow. 7
   2.4. Components Design 8
   2.5. Key Design Considerations 8
   2.6. API Catalogue 8

3. Data Design. 8
   3.1. Data Model 8
   3.2. Data Access Mechanism 9
   3.3. Data Retention Policies 9
   3.4. Data Migration 9

4. Interfaces 9

5. State and Session Management 10

6. Caching 10

7. Non-Functional Requirements 10
   7.1. Security Aspects 10
   7.2. Performance Aspects 10

8. References 11

1. Introduction

1.1. Scope of the document

This document explains the internal working of Edu Bot in detail. It covers each Python file, each important function, the data structures used, and how the web pages connect to backend. This document is the Low-Level Design (LLD) and goes deeper than the HLD.

1.2. Intended Audience

This document is written for:

- Internal guide and project evaluators
- Developers who want to understand or modify the Edu Bot code
- Students who want to learn how a Flask chatbot is built
- Testers who want to understand the system before testing

1.3. System Overview

Edu Bot is a keyword-based chatbot built using Python and Flask. It has two interfaces-a web interface runs and a CLI interface. The web interface runs on http://127.0.0.1:5000 and the CLI runs directly in terminal. Both interfaces use the same chatbot logic from chatbot.py. The project also has a dashboard page that shows usage statistics using the udemy_courses.csv dataset.

2. System Design

2.1. Module Design

- App.py – Main Flask server file. Handles all routes and connects web pages to backend logic.
- Chatbot.py – Contains the keyword matching logic. Takes student message and returns correct reply.
- Responses.py – Stores all keywords and answers in Python dictionary.
- Queries.json – Saves every student question with date, time, and category.
- Index.html – Main chat web page that the student uses.
- Dashboard.html – Analytics page showing chatbot usage statistics.
- Script.js – Handles chat behavior in the browser.
- Style.css – Controls how the web pages look.

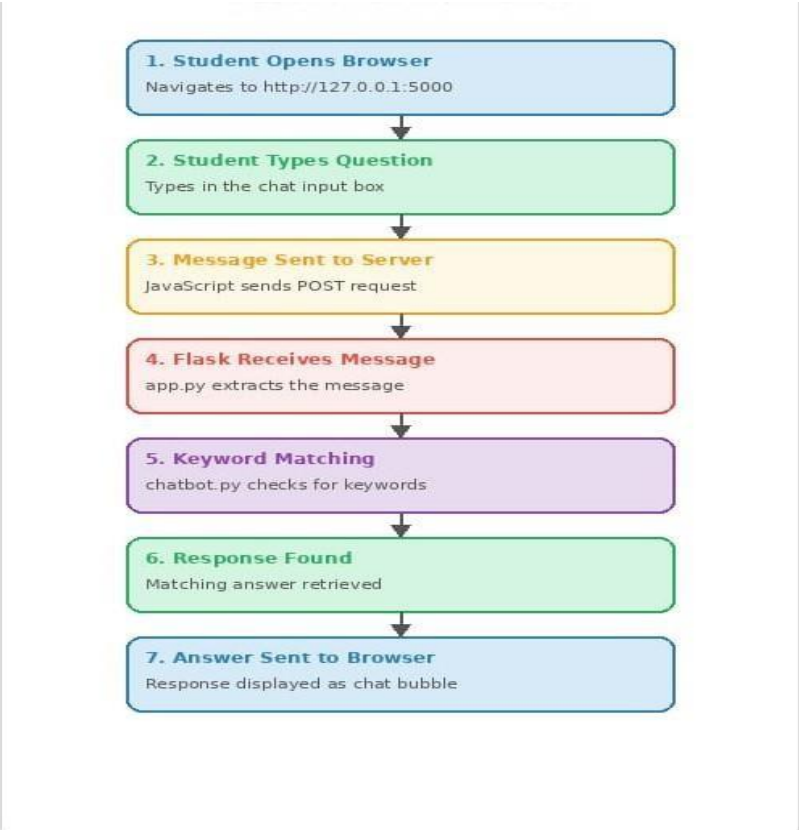
- Udemy_courses.csv – Dataset used for dashboard charts.

2.2. Function Design

The following are the main functions in Edu Bot and what they do:

- get_response(message) – This function is in chatbot.py. It takes the student's message as input, converts it to lowercase, checks for keywords, and returns the matching reply from responses.py.
- home() – This function is in app.py. It handles the GET request for the home page and loads index.html in the browser.
- Chat() – This function is in app.py. It handles the POST request from the chat page, calls get_response(), saves the query to queries.json, and returns the reply as JSON.
- Dashboard() – This function is in app.py. It handles the GET request for the dashboard page, reads udemy_courses.csv, and sends the data to dashboard.html.
- Save_query(question, category) – This function saves the student's question, category, date and time into queries.json.

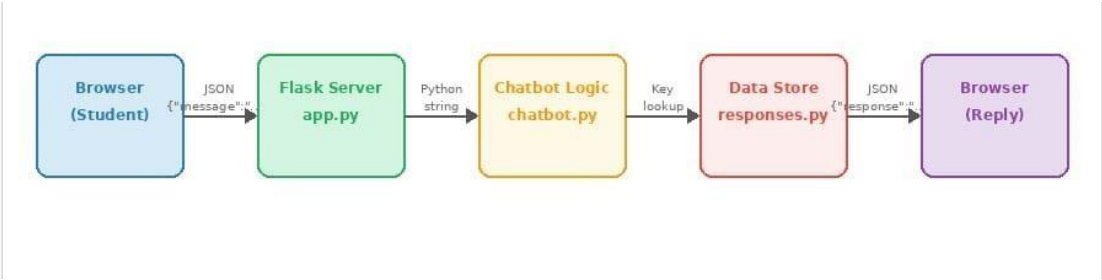
Function Flow Diagram:



2.3. Process Flow

1. Student opens the browser and goes to http://127.0.0.1:5000
2. Index.html loads and shows the chat interface
3. Student types a message and clicks **send**
4. Script.js sends the message to the /chat route using a POST request
5. App.py receives the message and passes it to get_response() in chatbot.py
6. Chatbot.py checks the message for keywords using responses.py
7. The matching reply is returned to app.py
8. App.py saves the question to queries.json and sends the reply back as JSON
9. Script.js receives the reply and displays it in the chat window

Process Flow Diagram:



2.4. Information Flow

The following points explain how information moves through Edu Bot:

- Student types a message in the browser – this is the input
- Script.js sends the message to app.py as a JSON object
- App.py extracts the message and passes it to chatbot.py
- Chatbot.py checks the message against responses.py and finds the reply
- The reply is sent back to app.py as a string
- App.py sends the reply to the browser as a JSON object
- Script.js displays the reply in the chat window as a chat bubble
- At the same time app.py saves the question details to queries.json

2.5. Component Design

The following table shows each component and its responsibility:

Component	Type	Responsibility
App.py	Python file	Starts Flask server and handles all routes
Chatbot.py	Python file	Contains keyword matching logic
Responses.py	Python file	Stores all keywords and answers
Queries.json	JSON file	Logs all student questions
Udemy_courses.csv	CSV file	Dataset for dashboard charts
Index.html	HTML file	Main chat web page
Dashboard.html	HTML file	Analytics dashboard page
Script.js	JS file	Handles chat behavior in browser
Style.css	CSS file	Controls styling of web pages

2.6. API Catalogue

Edu Bot has the following API end points:

Endpoint	Method	Description
/	GET	Loads the main chat page index.html
/Chat	POST	Receives student message and returns chatbot reply
/Dashboard	GET	Loads the analytics dashboard page
/Api/stats	GET	Returns usage statistics for the dashboard charts

3. Data Design

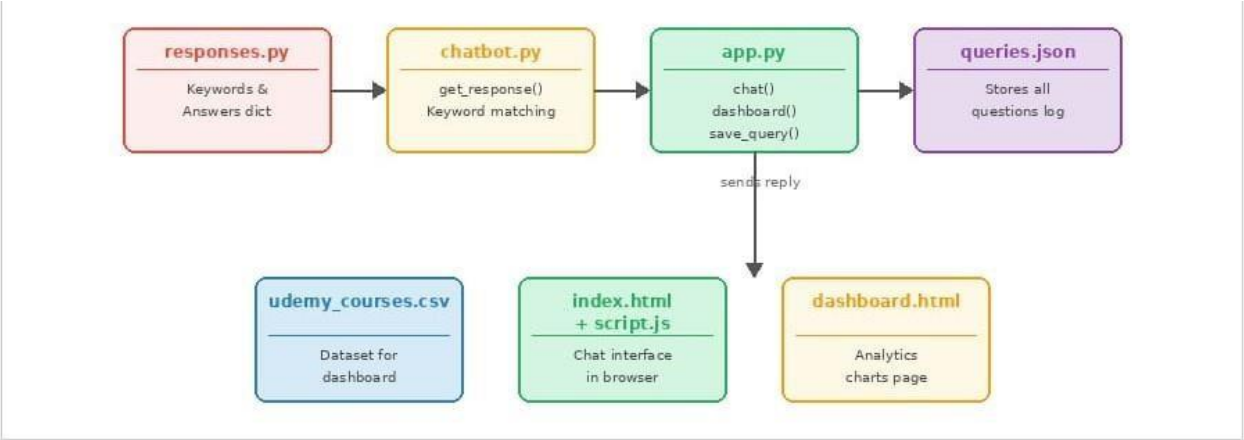
3.1. Data Model

Edu Bot uses three data sources instead of a database:

- Responses.py - A Python dictionary where each key is a keyword and each value is the chatbot reply. Example: “fee” maps to the fee information reply.

- Queries. Json - A JSON file that stores a list of objects. Each object has four fields: question, category, date, and time.
- Udemy_courses.csv - A CSV file that containing Udemy course information like title, price, subscribers, and rating.

Data Flow Diagram:



3.2. Data Access Mechanism

Edu Bot access data in the following ways:

- Responses.py is imported directly into chartbot.py. The get_response() function loops through the dictionary keys to find a matching keyword.
- Queries.json is opened using Python's built in open () function. Every time a student asks a question, the new query is appended to the JSON list and saved back to the file.

Udemy_courses.csv is read using the panda's library inside app.py. The data is converted to a list and sent to dashboard.html for displaying charts.

3.3. Data Retention Policies

The following points explain how data is handled in Edu Bot:

- No personal information about students is stored anywhere in the system
- Queries.json only stores the question text, category, date and time

- The chat history on the web page is stored only in the browser memory and disappears when the page is refreshed
- Udemy_courses.csv is never modified by the system – it is only read
- There is no login system so no passwords or user details are stored

3.4. Data Migration

At this stage, data migration is not needed because Edu Bot uses simple files to store data. There is no database to migrate.

If the project is upgraded in the future, the following migrations can be done:

- Responses.py dictionary can be moved to a SQLite or MySQL database table
- Queries.json can be moved to a database table with columns for question, category, date and time
- Udemy_courses.csv can be imported into a database table for faster querying

4. Interfaces

4.1. Web Interface

The web interface is built using HTML, CSS and JavaScript. It runs in the browser http://127.0.0.1:5000. It has following elements:

- Navigation bar – shows the Edu Bot logo and a button to open the dashboard page
- Course cards panel – shows a list of available courses on the left side
- Chat panel – shows the chat window on the right side where students type their questions
- Input box – where the student types their message
- Send button – sends the message to the Flask backend
- Chat bubbles – shows student messages on the right and bot replies on the left • Footer – shows the student name and technologies used

4.2. Dashboard Interface

The dashboard page opens at http://127.0.0.1:5000/dashboard. It shows the following elements:

- Total queries counter – shows the total number of questions asked by students
- Category chart – a bar chart showing which topics were asked about the most
- Daily usage chart – a line chart showing how many questions were asked each day
- Top keyword table – a table showing the most common keywords used by students
- Udemy statistics – shows total courses, paid vs free count, average price and average rating from the udemy_courses.csv dataset

4.3. CLI Interface

The CLI interface runs directly in the terminal. The student runs the chatbot.py file using the command python chatbot.py. It works in the following way:

- The terminal shows a welcome message when the program starts
- The student types a question and presses Enter
- The chatbot checks the question for keywords and prints the reply
- The student can keep asking questions in a loop
- The student types exit or quit to stop the program
- No browser or internet connection is needed to use the CLI version

5. State and Session Management

5.1. Session Handling

Edu Bot does not use any server side session management. The following points explain how state is handled:

- Each message sent by the student is treated as a completely new and independent request
- The chat history shown on the web page is stored only in the browser using JavaScript variables
- When the page is refreshed the chat history disappears
- The Flask server does not remember any previous conversations
- There is no login system so there is no user session to manage

5.2. State Management

The following points explain how state is managed in Edu Bot:

- A question counter variable is kept in memory inside app.py while the server is running
- This counter increases by one every time a student asks a question
- The counter resets to zero when the Flask server is restarted
- queries.json acts as a permanent state store — it keeps a record of all questions even after the server restarts

The udemy_courses.csv data is loaded into memory when the dashboard page opens and is cleared when the page closes

6. Caching

6.1. Caching Strategy

Edu Bot does not use any special caching system. The following points explain how caching works in the current version:

- The responses.py dictionary is loaded into memory when the server starts so every keyword lookup is very fast and does not need caching
- The browser automatically caches static files like style.css and script.js so the page loads faster on repeat visits
- The udemy_courses.csv file is read from disk every time the dashboard page opens — in a future version this can be cached in memory to make the dashboard load faster
- No Redis or Memcached or any other caching tool is used in the current version

7. Non – Functional Requirements

7.1. Security Aspects

The following security points apply to Edu Bot:

- All student input is treated as plain text — no HTML or script tags are executed
- No personal data like name, email or device details are collected or stored

- The application runs locally on the student's computer and is not exposed to the internet
- There is no login system so there is no risk of password leaks
- The queries.json file only stores question text, category, date and time

No sensitive information is stored anywhere in the project files

7.2. Performance Aspects

The following points explain the performance of Edu Bot:

- The chatbot responds very quickly because all answers are stored in memory in responses.py and no database query is needed
- A small artificial delay of 0.5 to 1.5 seconds is added in the web interface to make the chat feel more natural
- The keyword matching logic is simple and fast — it loops through the dictionary keys and returns the first match
- The dashboard loads the udemy_courses.csv file each time it opens — this takes a small amount of time but is acceptable for the current scale
- The system is designed for single user local use and performs well for demo and academic purposes
- Flask development server is used which is suitable for small scale use

8. Reference

1. README.md
2. App.py
3. Chatbot.py
4. Responses.py
5. Queries.py
6. Udemy_courses.csv
7. Templates/index.html
8. Templates/dashboard.html
9. Static/style.css
10. Static/script.js